Atividade 2 (Vale 50% da Nota 2)

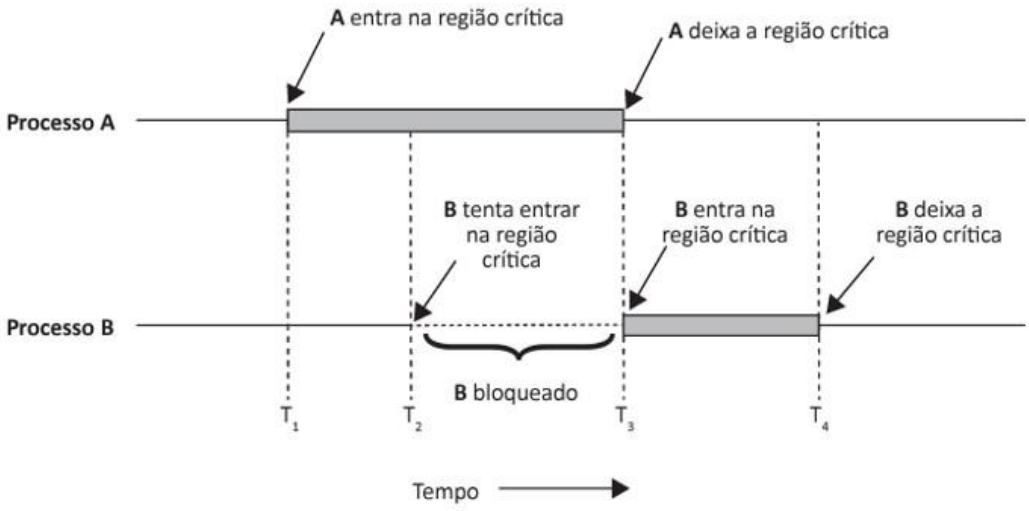
Nome: Mateus Dassoler Spezia / Vinícius Oneda

Exercício 1

Durante parte do tempo, um processo está ocupado realizando computações internas e outras coisas que não levam a condições de corrida. No entanto, às vezes, um processo tem de acessar uma memória compartilhada ou arquivos, ou realizar outras tarefas críticas que podem levar a corridas. Essa parte do programa onde a memória compartilhada é acessada é chamada de região crítica ou seção crítica. Se conseguíssemos arranjar as coisas de maneira que dois processos jamais estivessem em suas regiões críticas ao mesmo tempo, poderíamos evitar as corridas. Embora essa exigência evite as condições de corrida, ela não é suficiente para garantir que processos em paralelo cooperem de modo correto e eficiente usando dados compartilhados. Precisamos que quatro condições se mantenham para chegar a uma boa solução.

1. Dois processos não podem estar simultaneamente em suas respectivas regiões críticas.
2. Não se deve assumir nada quanto à velocidade de execução ou à quantidade de CPUs disponíveis.
3. Um processo que esteja fora de sua região crítica não pode impedir a execução de outro processo.
4. Nenhum processo deve aguardar indefinidamente para conseguir entrar em sua região crítica.

Em um sentido abstrato, o comportamento que queremos é mostrado na figura a seguir.



Considerando o texto e a figura apresentados, analise as assertivas a seguir e a relação entre elas.

I. Em determinadas situações, a exclusão mútua pode ser alcançada por meio da desativação de interrupções, recurso que é controlado pelo Sistema Operacional, não sendo permitido ao usuário realizar esse controle.

PORQUE

II. A desativação de interrupções consiste em uma técnica capaz de evitar que o processador, enquanto executa um processo em sua região crítica, seja interrompido para executar outro código, sendo considerada mais eficiente em sistemas multiprocessadores devido à quantidade de processos concorrentes.

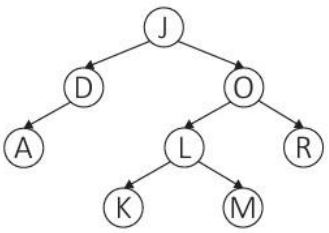
Com base nessas assertivas, assinale a alternativa correta.

A. As assertivas I e II são verdadeiras, e a II justifica corretamente a I.
B. As assertivas I e II são verdadeiras, porém a II não justifica corretamente a I.
C. A assertiva I é verdadeira, enquanto a II é falsa.
D. A assertiva I é falsa, enquanto a II é verdadeira.
E. As assertivas I e II são falsas.

Exercício 2

A estrutura de dados denominada Árvore Binária de Busca constitui uma técnica fundamental na programação. Uma árvore binária é definida como um conjunto finito de elementos que pode estar vazio ou ser dividido em três subconjuntos: (1) a raiz, elemento inicial e único; (2) a subárvore esquerda, que isoladamente também configura uma árvore; e (3) a subárvore direita, que igualmente, de forma isolada, constitui outra árvore. É possível que a árvore não possua elementos, caracterizando uma árvore vazia. Sua definição é recursiva e, por isso, diversas operações realizadas sobre árvores binárias utilizam recursão. Considerando "A" como a raiz de uma árvore binária e "B" como a raiz de sua subárvore esquerda ou direita, afirma-se que "A" é pai de "B" e que "B" é filho de "A". Um elemento que não possui filhos é denominado folha. A altura da árvore corresponde ao número de elementos presentes no caminho descendente mais longo que liga a raiz a uma folha.

A Árvore Binária de Busca é uma especialização da árvore binária, na qual o valor armazenado no filho esquerdo é numericamente menor que o valor do elemento pai. De modo análogo, o valor armazenado no filho direito é numericamente maior ou igual ao valor do elemento pai. O principal objetivo da organização dos dados em Árvores Binárias de Busca é facilitar a busca por elementos específicos. O percurso completo de uma árvore binária consiste em visitar todos os seus elementos conforme determinado critério, a fim de processá-los. As três formas clássicas de percurso são: (1) pré-ordem, (2) em-ordem e (3) pós-ordem. A figura a seguir apresenta um exemplo de árvore binária.



Exemplo de árvore binária

Considerando o texto e a figura apresentados, e supondo que a lista de elementos numéricos (27, 34, 40, 18, 23, 5, 25, 36, 10, 7, -2) seja inserida, elemento por elemento, em uma Árvore Binária de Busca inicialmente vazia, seguindo a ordem da esquerda para a direita, assinale a alternativa correta.

A. A árvore resultante apresentará 5 níveis de altura, contendo 6 elementos à esquerda da raiz principal (inicial) e 4 elementos à direita.

B. No percurso em pré-ordem, os elementos serão visitados na seguinte sequência (do primeiro ao último): -2, 7, 10, 5, 25, 23, 18, 36, 40, 34, 27.

C. No percurso em ordem, os elementos serão processados na seguinte sequência (do primeiro ao último): -2, 5, 7, 10, 18, 23, 25, 27, 34, 36, 40.

D. No percurso em pós-ordem, os elementos serão visitados na seguinte sequência (do primeiro ao último): 27, 18, 5, -2, 10, 7, 23, 25, 34, 40, 36.

E. O número máximo de elementos que essa árvore pode conter, considerando 10 níveis, é de 1.024 elementos.

Exercício 3

Há uma grande variedade de implementações de algoritmos de ordenação. Um dos aspectos a serem analisados, por exemplo, é a quantidade máxima e média de comparações necessárias para ordenar um vetor com n elementos. Além disso, considerase que um algoritmo de ordenação é estável quando mantém a ordem relativa entre elementos iguais; ou seja, esses elementos aparecem na sequência ordenada na mesma ordem em que estavam na sequência original. Considere o algoritmo a seguir, no qual A representa um vetor e i, j, Io e hi correspondem aos índices desse vetor:

```
algoritmo ordena(A, lo, hi)
    se lo < hi então
       p := particao(A, lo, hi)
       ordena(A, lo, p - 1)
       ordena(A, p + 1, hi)


algoritmo particao(A, lo, hi)
    pivot := A[hi]
    i := lo
    repita para j := lo até hi
       se A[j] < pivot entao
       troca A[i] com A[j]
       i := i + 1
    troca A[i] com A[hi]
    return i
```

Com base no algoritmo apresentado, analise as seguintes afirmações:

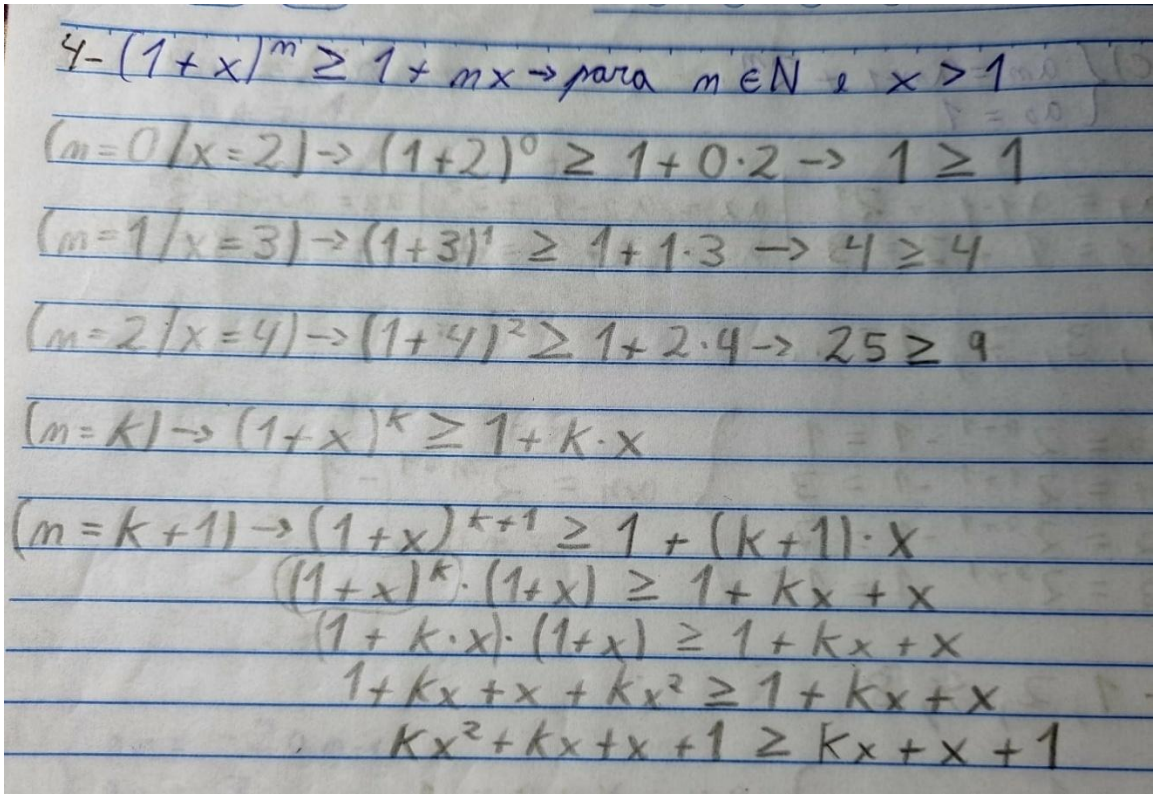
I. O algoritmo requer um espaço adicional de ordem O(n) devido à pilha de recursão. II. O algoritmo apresentado é um método de ordenação recursivo e estável. III. Em média, o algoritmo necessita de O(n log n) comparações para ordenar n elementos. IV. Utilizar o primeiro elemento do vetor como pivô é mais eficiente do que utilizar o último.

Está correto apenas o que se afirma em:

A) I e III.
B) II e IV.
C) III e IV.
D) I, II e III.
E) I, II e IV.

Exercício 4

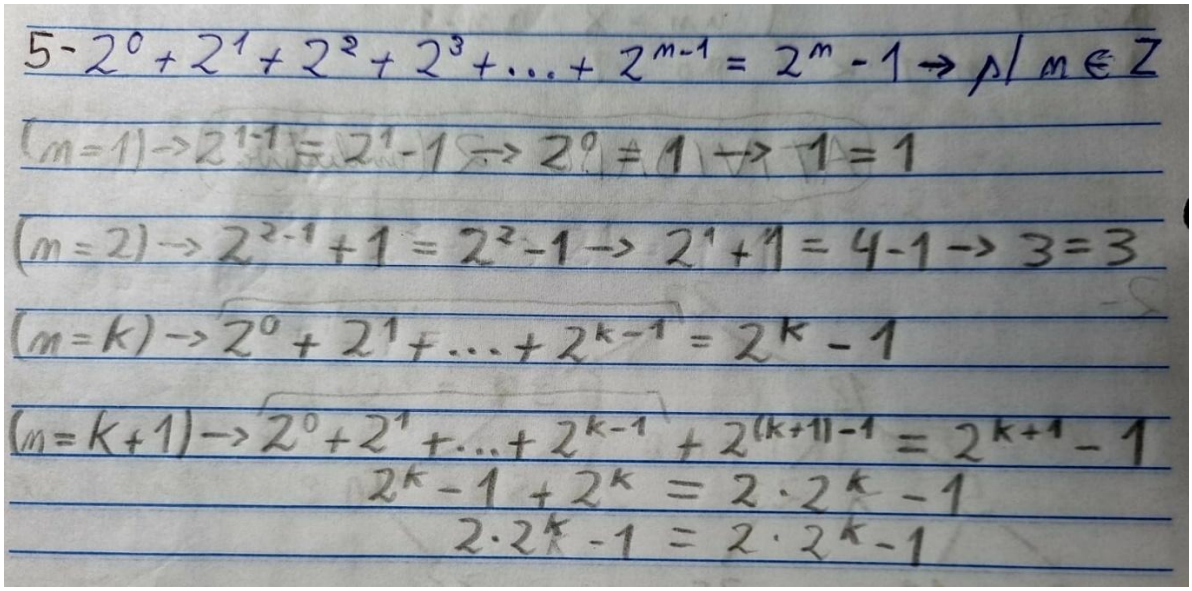
Uma desigualdade interessante da matemática é chamada desigualdade de Bernoulli. Esta Desigualdade afirma o seguinte: Sejam $n \in N$ e $x > 1$. Então $(1 + x)^n \geq 1 + nx$. Use o princípio de indução matemática para demonstrar a veracidade dessa desigualdade.



Exercício 5.

Use a Indução Matemática para demonstrar que os resultados são válidos para qualquer inteiro positivo n.

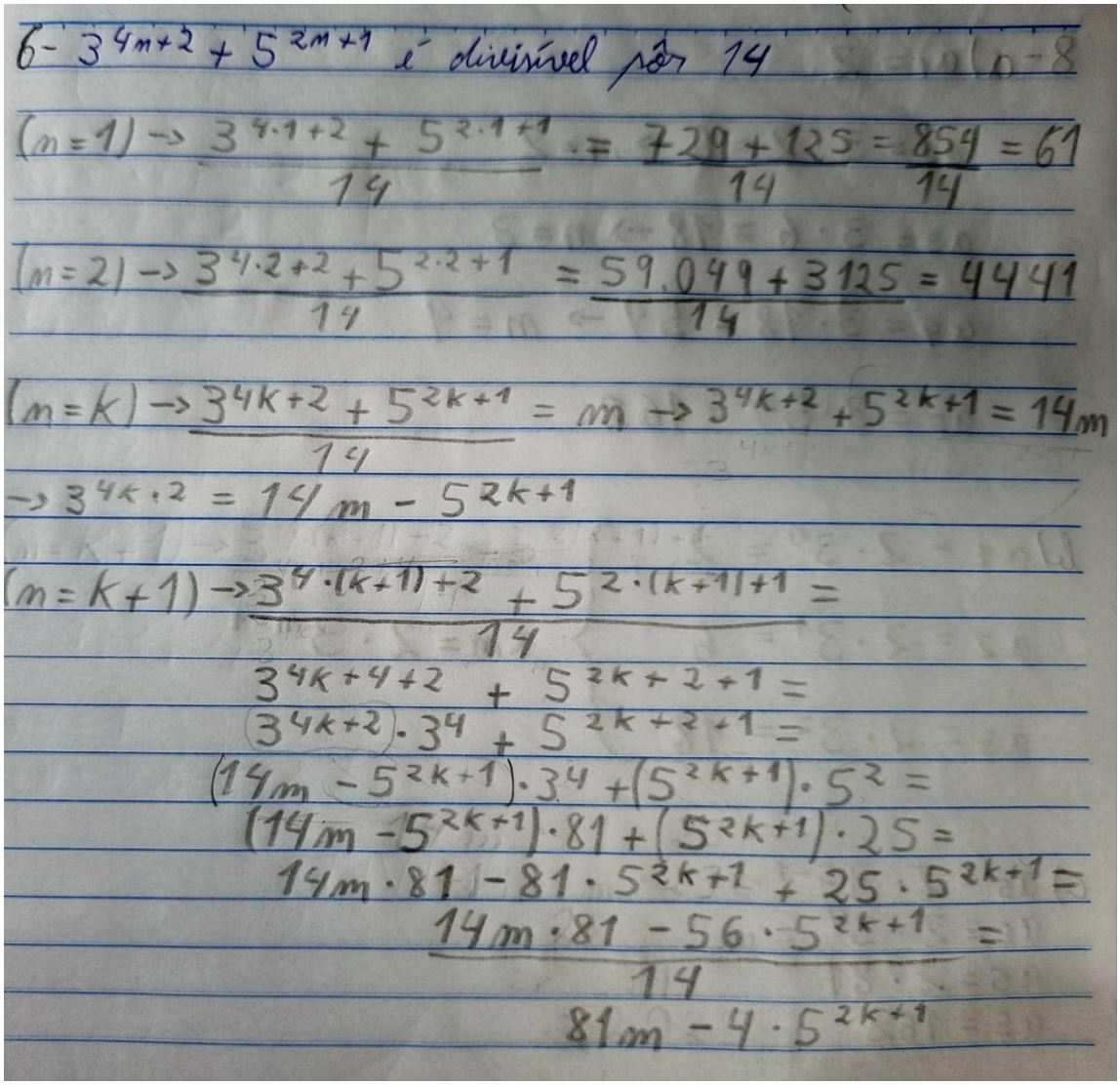
$2^0 + 2^1 + 2^2 + 2^3 + \ldots + 2^{n-1} = 2^n - 1.$



Exercício 6.

Prove que para qualquer inteiro n,

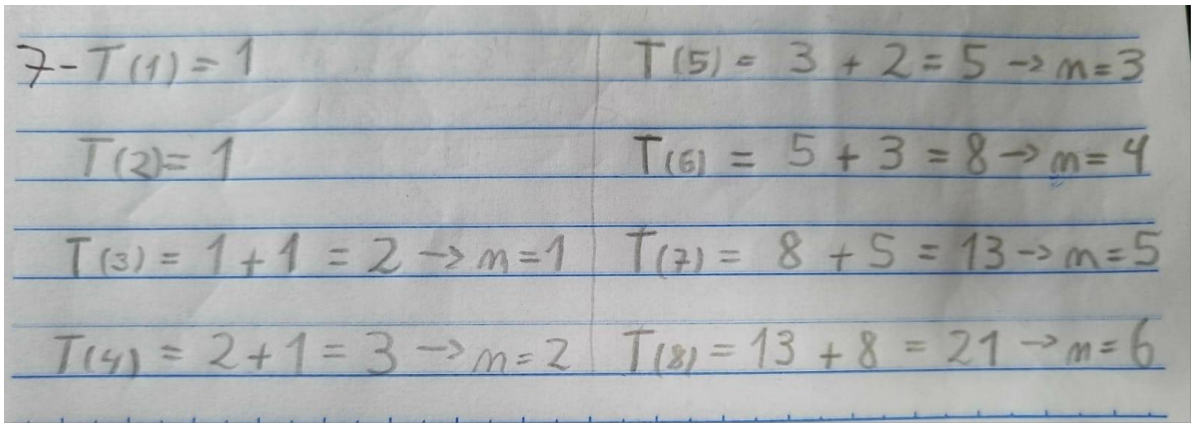
O número $3^{4n+2} + 5^{2n+1}$ é divisível pôr 14.



Exercício 7

a) Dada a lei de formação recorrente abaixo determine os oito primeiros termos da sequência.

$$\begin{cases} T_{n+2} = T_{n+1} + T_n \\ T_{(1)} = 1 \\ T_{(2)} = 1 \end{cases}$$



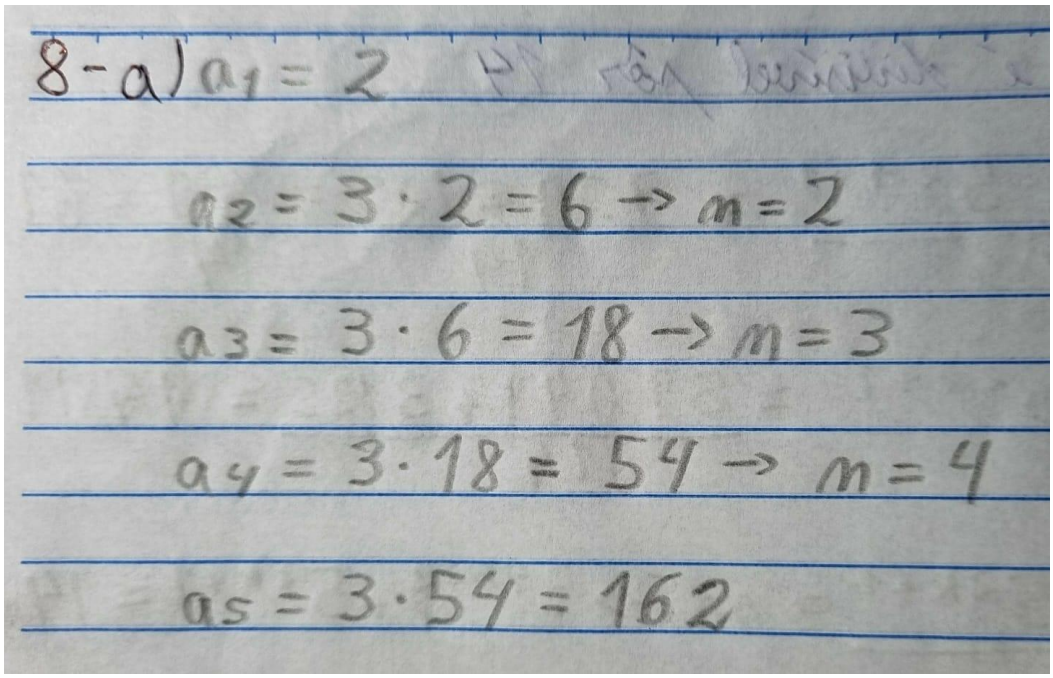
b) Está sequência recebe o nome de: **Fibonacci.**

8) Considere a sequência definida pela relação de recorrência:

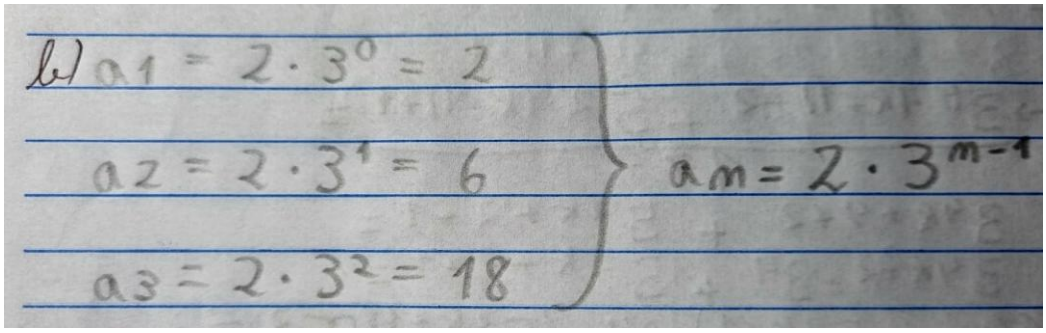
$$\begin{cases} a_n = 3a_{n-1} \\ a_1 = 2 \end{cases}$$

com $n \in \mathbb{N}$ e $n \geq 2$.

a) Calcule os cinco primeiros termos da sequência.



b) Determine uma fórmula fechada para o termo geral a_n.



c) Verifique se a fórmula encontrada no item anterior gera corretamente o 5º termo da sequência.

